PIPDESK · GO-LIVE REVIEW

Ready to trade live on FundingPips?

Review of the order and risk path, MT5 reliability and architecture of Pipdesk, a self-hosted charting and order-entry app for a FundingPips MetaTrader 5 account, before it goes live on Windows.

VERDICT

Not ready yet. The architecture is sound and well tested against the simulator. But five Critical issues can let the account breach FundingPips' rules or place an order the trader did not intend, and none of them show up in mock mode. Most fixes are small and contained.

32 findings by severity

Critical	High	Medium	Low
5	7	15	5
Could place an unintended order, bypass a risk rule or breach the account	Wrong numbers, a silent failure in the order path, or a likely live-only failure	Reliability or maintainability problems that will cause bugs	Polish

CONTENTS

1. Executive summary
2. Scope and method
3. Baseline results
4. Architecture
5. Key flows
6. Review by area
7. Findings (all 32)
8. Probe evidence
9. Top 5 changes
10. Improvement plan
11. Pre-live checklist
12. What could not be determined

Appendices: A · Test coverage map B · Mock versus real MT5 C · Large files

Date	12 October 2026
Codebase	The `Trading Platform` folder as of 12 October 2026 (not under version control, so there is no commit ID): about 12,400 lines across a Python backend (3,538 source + 1,552 test lines) and a React frontend (6,648 source + 708 test lines)
Stack	Python, FastAPI, Pydantic v2, SQLModel on SQLite, MetaTrader5 package (Windows) or `mt5_mock.py`; React 19, TypeScript, Vite, Tailwind 4, lightweight-charts 5, Vitest
Status	Understanding, review, prioritisation and plan complete. **No code was changed.** Fixes wait for the owner's approval.
Prepared by	Claude (AI code reviewer) at the owner's request. Findings cite file and line; behaviour that needs a real MT5 terminal is marked plausible.

1 Executive summary

What is solid

- **Clean layering.** `risk.py` is a pure rules engine with no I/O; `risk_service.py` connects it to MT5 and SQLite; `trading.py` runs each action as lock → risk check → `order_send` → journal; `mt5_bridge.py` is the only module that talks to MT5.
- **One lock** spans the risk check through `order_send`, every order action goes through it, and `order_send` is never retried by the backend or the frontend.
- **Risk-reducing actions are exempt** from every rule inside the engine: closing, cancelling, adding a stop and moving a stop closer.
- **The security model works as designed:** Host allow-list for HTTP and WebSocket, origin checks, a required header on every state-changing request, and credentials held as `SecretStr`, never logged and never returned.
- **A good test base:** 172 backend and 84 frontend tests, all passing; the risk rules are covered in depth, including boundaries.

What blocks going live

- **F1 · Day anchor can reset mid-day.** Today's losses are forgotten. Probe P1: down 4% at 13:00 server time, reset changed to 14:00 → daily floor drops from $95,000 to $91,200, and it re-anchors again at 14:00.
- **F2 · Risk state not tied to the account.** Probe P3: same-day switch from a $10k to a $100k account → daily floor $9,500, drawdown floor $9,000. Hits demo → FundingPips and Phase 1 → Phase 2; the app also trades whatever account the terminal shows. The badge can't tell MetaQuotes-Demo from a demo-type prop account.
- **F3 · Late day anchor uses current values.** Probe P6: app started at 13:00 after a 3% loss taken in the terminal → daily floor $92,150 instead of FundingPips' $95,000. Overnight floating profit and weekend gaps also drop out of the base.
- **F4 · Unknown outcomes shown as failures.** Probes B3 and B4: the position was open while the API returned 500 (no journal row) or 503 (journal says error). Clicking the still-open button opens a second position.
- **F5 · One-click double submit.** An unintended duplicate position. Risk limits still apply, but see F8.

Most likely to break or weaken the first live run

- **F6 · Closing depends on the full context.** Probe B2: with `SERVER_TIME_SYMBOL` missing on the broker, closing a position returns 404 and Flatten all closes nothing.
- **F7 · Empty results may come back as None.** Probe B1: `GET /positions` and every order return 502; the stream sends only account and status (no ticks, candles or positions). Fails closed, but the app is unusable. Plausible: verify on Windows.
- **F8 · Just-sent orders can be invisible.** Max positions, max lots and the worst case can be exceeded by orders placed in quick succession (see F5). Plausible.

Recommended order of work

1. **Step 0:** put the project under git with a baseline commit (needs approval).
2. **Quick wins QW1–QW5:** day key, never-block independence, one-click guard, empty results, in-flight exposures. All small, all tighten-only, each with tests written first.
3. **Decisions** on F2, F3, F4, F5, F9, F10 and F20, then implement them.
4. **Remaining quick wins** QW6–QW13.
5. **The pre-live checklist** (section 11) on a demo account, then again before the first FundingPips session.
6. **Structural refactors** S1–S6, each behind tests that pin current behaviour.

2 Scope and method

- **Read every source and test file** (12 backend modules, 10 backend test files, 44 frontend files), traced five flows end to end, and mapped module dependencies on both sides.
- **Ran the four checks** the project defines; results are reproduced verbatim in section 3.

- **Confirmed suspected issues with probes:** throwaway scripts that import the real backend modules and drive them against the mock, with an in-memory SQLite database. The scripts lived in a scratch folder, outside the project. Results are in section 8.
- **Constraints kept:** no code changes; the app was not started; `backend/.env` was never opened; `trading.db` was not touched; no order was sent to any MT5 account.
- **Labels:** *confirmed* means reproduced by a probe or evident in the code; *plausible* means it depends on real-MT5 behaviour that cannot be observed on the Mac.
- **Paths:** backend files are in `backend/` (tests in `backend/tests/`); frontend files are in `frontend/src/` (`components/`, `lib/`, `api/`, `hooks/`). References read `file:first-last` line. In tables, component, hook and test files are cited by file name alone, since every name is unique.

Severity and effort

SEVERITY	MEANING
Critical	Could place an unintended order, bypass a risk rule or breach the account
High	Wrong numbers, a silent failure in the order path, or a likely live-only failure
Medium	Reliability or maintainability problems that will cause bugs
Low	Polish

Effort: **S** is a small, contained change; **M** touches several modules or needs a design decision; **L** is a multi-step refactor.

3 Baseline results

Run on the Mac on 12 October 2026, before any change. Output is reproduced verbatim (blank lines trimmed).

```
$ python -m pytest
```

```
.....................................................................  [ 41%]
.....................................................................  [ 83%]
.............................                                          [100%]
============================== warnings summary ==============================
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1
  /Users/bipin/Projects/Trading Platform/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1:
  StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2`
instead.
    from starlette.testclient import TestClient as TestClient  # noqa
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
172 passed, 1 warning in 1.10s
```

```
$ cd frontend && npm test
```

```
 RUN  v5.0.3 /Users/bipin/Projects/Trading Platform/frontend
 Test Files  8 passed (8)
      Tests  84 passed (84)
   Start at  01:45:58
   Duration  129ms (transform 64%, import 19%, tests 11%, worker 6%)
```

```
$ npm run typecheck
```

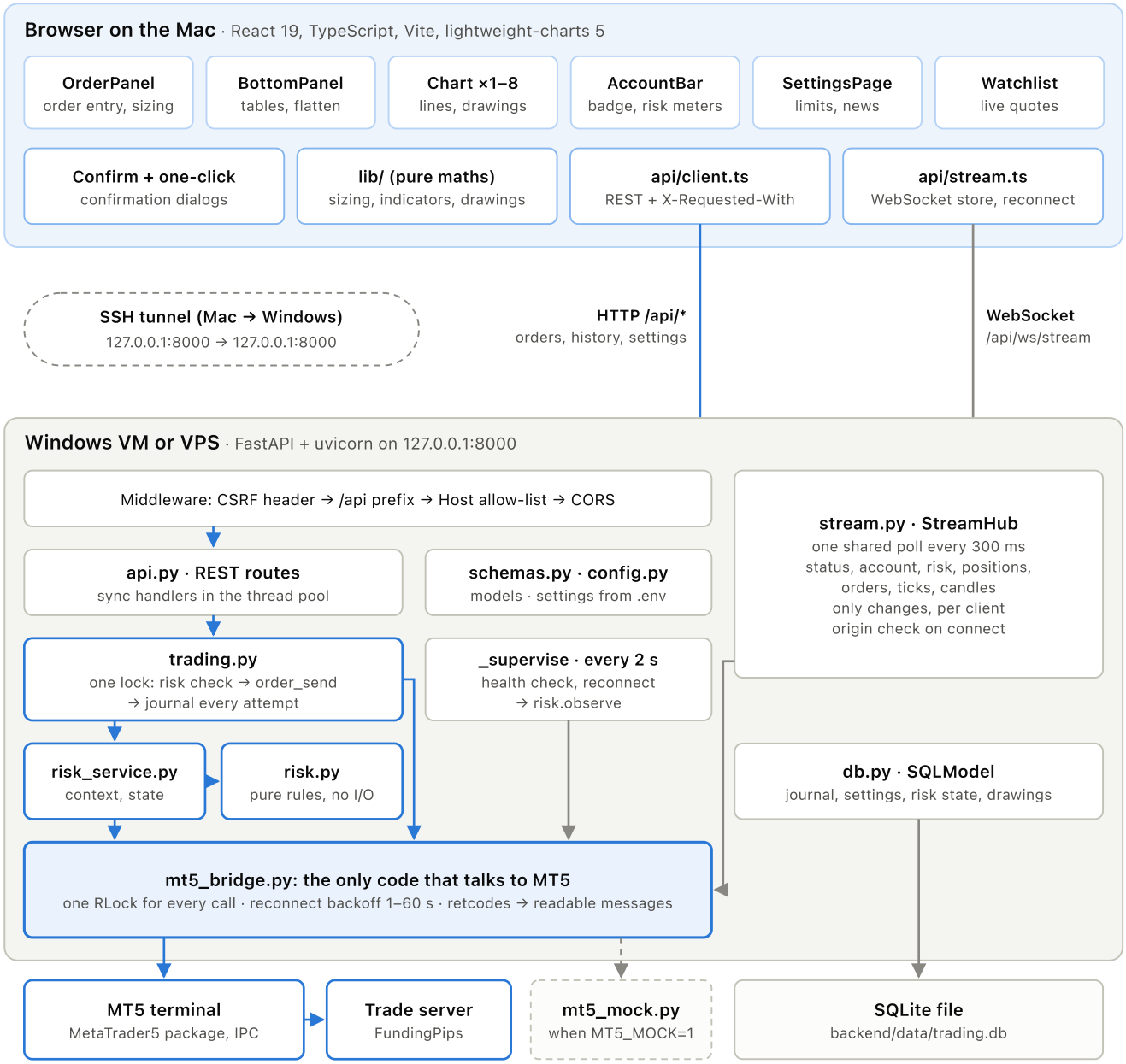
```
> tsc --noEmit -p .          (exit 0, no output)
```

```
$ npm run build
```

```
vite v8.3.4 building client environment for production...
✓ 55 modules transformed.
dist/index.html                   0.65 kB │ gzip:   0.42 kB
dist/assets/index-CyoEM7kO.css   23.08 kB │ gzip:   5.23 kB
dist/assets/index-4eoneMji.js   523.05 kB │ gzip: 163.79 kB
✓ built in 92ms
(!) Some chunks are larger than 500 kB after minification.           (exit 0)
```

Also noted: no linter or formatter is configured (no ruff, mypy or eslint; nothing installed). The Mac virtual environment runs Python 3.14.6, while the README specifies 3.12 for Windows (F26). The build rewrote `frontend/dist`, which is git-ignored build output.

4 Architecture



Blue is the order path: every order action takes one lock, passes the risk engine, calls `order_send` through the bridge and is journaled. **Grey** is live data and supervision. **Dashed** marks the SSH tunnel and the simulator used on the Mac.

Components

LAYER	MODULES	RESPONSIBILITY
Frontend · UI	OrderPanel, BottomPanel, Chart (TradeLines, DrawingLayer, IndicatorSet), AccountBar, SettingsPage, Watchlist	Order entry and sizing; tables, close and flatten; charts with draggable trade lines, drawings and indicators; account and risk meters; settings and news
Frontend · shared	`api/client.ts`, `api/stream.ts`, `Confirm.tsx`, `lib/*`	REST with the CSRF header; WebSocket store with reconnect; confirmation and one-click; pure maths (indicators, sizing, drawings)
Backend · API	`api.py`, `schemas.py`, `config.py`	Routes, middleware, error mapping, request and response models, settings
Backend · order path	`trading.py`, `risk_service.py`, `risk.py`	Lock → risk check → `order_send` → journal; context and persistence; pure rules

LAYER	MODULES	RESPONSIBILITY
Backend · live data	`stream.py`, `_supervise` in `api.py`	One 300 ms poll diffed per client; 2 s health check and risk observation
Backend · MT5	`mt5_bridge.py`, `mt5_mock.py`	The only MT5 touchpoint (one lock, backoff, readable retcodes); simulator for `MT5_MOCK=1`
Storage	`db.py` → SQLite `backend/data/trading.db`	Tables keyvalue (risk settings, risk state, drawings), newsevent and journalentry

Integrations and dependencies

- **MT5:** only `mt5_bridge.py:154–166` loads the MetaTrader5 package or the mock. No other module touches MT5; only tests import the mock directly.
- **Broker:** reached only through the MT5 terminal. Live mode requires Windows next to a running terminal with Algo Trading enabled.
- **Browser:** REST under `/api/*` (an alias of `/*`) and one WebSocket, over an SSH tunnel from the Mac. The backend serves the built UI on the same port.
- **Where the frontend recomputes money or risk its own way:** `lib/trading.ts:44–73` sizes with a `tick_value` snapshot while the backend uses `order_calc_profit`; chart-line P/L labels (`lib/tradeLines.ts:38–42`); two pip-size rules (`lib/trading.ts:16–21` versus `components/Chart.tsx:231–232`); "Today's P/L" (`components/AccountBar.tsx:121`); and the SL-or-TP guess for close toasts (`hooks/useTradeEvents.ts:38–42`).

5 Key flows

5.1 Placing an order

1. The order panel derives entry, SL, TP and volume on every render (`components/OrderPanel.tsx:117–162`) and runs a debounced dry run, `POST /risk/check` (`:164–193`).
2. `review()` rounds price, SL and TP to the symbol's digits (`:239–241`); `confirm()` shows the dialog, or runs at once in one-click mode (`components/Confirm.tsx:56–68`).
3. `run` re-reads the live quote for market orders (`components/OrderPanel.tsx:265–266`) and calls `trade()` → `request()`, which adds `X-Requested-With` (`api/client.ts:52–78`).
4. Middleware (`api.py:115–128`) passes it to `TradingService.place` (`trading.py:92–120`).
5. `_execute` takes the lock (`trading.py:51`); `RiskService.evaluate` builds the context (`risk_service.py:46–64`) and runs `RiskEngine.evaluate` (`risk.py:333–475`).
6. The bridge's `market_order` or `pending_order` calls `order_send` (`mt5_bridge.py:483–559`); every attempt is journaled (`trading.py:55–74`); the response is 200, 409 or 422 (`api.py:138–147`).
7. The next poll (within 300 ms) streams `positions`; chart lines (`components/useTradeLines.ts:37–39`) and the Positions table update.

5.2 Changing or closing a trade from the chart

1. Pointer capture and hit-testing in `components/TradeLines.ts:143–218`; `onDrop` (`components/useTradeLines.ts:87–162`) always confirms, and always sends **both** SL and TP, so a TP is never wiped by accident.
2. `PATCH /positions/{t}` → `trading.py:122–127`; adding or tightening a stop returns allowed early (`risk.py:373–380`); the bridge sends `TRADE_ACTION_SLTP` (`mt5_bridge.py:567–576`).
3. Partial close: `components/BottomPanel.tsx:145–192` → `trading.py:138–144`; the engine always allows closes (`risk.py:342–343`).
4. Flatten all cancels pending orders first, then closes positions, each through `_execute` (`trading.py:152–169`). The context built before the short-circuit is the gap described in F6.

5.3 Live stream

1. `connect()` re-sends subscriptions after every reconnect (`api/stream.ts:91–118`).
2. The server checks the origin (`api.py:349–356`) and validates each message (`stream.py:77–92`).
3. One shared poll builds a snapshot in a worker thread (`stream.py:96–153`) and sends each client only what changed (`:155–177`).
4. `cache()` and `useStream` feed components (`api/stream.ts:120–153`); candles go through `mergeLive` (`components/Chart.tsx:271–288`).

5.4 Risk state over time

1. At startup, settings and state load from SQLite (`risk_service.py:38–41`).
2. Every 2 s the supervisor calls `observe` (`api.py:68–79`). The day key comes from the server-time symbol's tick time or an IANA zone (`risk.py:201–211`).
3. A new key records the anchor (flagged late after 300 s), the high-water mark updates, and the observed initial balance is set once (`risk.py:213–227`); state is saved when it changes (`risk_service.py:69–71`).
4. Meters come from `status()` (`risk.py:263–299`), streamed as `risk` and drawn by `components/AccountBar.tsx:132–147`.

5.5 Charts and drawings

1. History loads on a symbol or timeframe change and after a reconnect (`components/Chart.tsx:237–266`).
2. Within 20 bars of the left edge, `/candles?before=` pages back via `copy_rates_from(before − 1)`, filtered to strictly older bars (`mt5_bridge.py:371–391`).
3. Indicators are pure functions, recomputed on each update (`components/IndicatorSet.ts:86–104`).
4. Drawings live in one page-wide store and save with a 600 ms debounce through `PUT /drawings/{symbol}` (`lib/drawingStore.ts:88–117`), validated by `schemas.py:345–391`; charts of the same symbol share them, or are filtered by pane when sync is off (`components/useDrawings.ts:36–55`).

6 Review by area

6.1 Risk and trading correctness

VERIFIED

- Every order action (place, modify order, cancel, modify position, close, flatten) goes through `TradingService._execute`, which holds `RiskService.lock` from `evaluate` through `order_send` (`trading.py:44–76`). No other module calls the bridge's trading methods.
- The worst case sums every open stop, every pending order filled then stopped, and commission (`risk.py:439–470`).
- `order_send` is called once per action; neither `mt5_bridge.py:311–324` nor `api/client.ts:52–78` retries.
- A disconnected terminal fails closed: calls raise `MT5Unavailable` until reconnected (`mt5_bridge.py:313–314`). Money comparisons use explicit tolerances (`risk.py:40`).

ISSUES

F1–F5 and F8–F11, F13, F17, F18. Price and volume rounding, stops level and freeze level: F11 (freeze level is not exposed at all). Filling mode and deviation: F15, F16, F30. Pip rules: FX, JPY and gold follow common convention; silver at 0.1 is unusual; the chart tools disagree (F21).

6.2 MT5 integration and reliability

VERIFIED

- Every MT5 call is serialised behind one lock (`mt5_bridge.py:214`). Sync routes run in the thread pool, and the stream and supervisor use `run_in_threadpool`, so nothing blocks the event loop.
- Reconnect backs off exponentially from 1 s to 60 s, with an immediate first retry (`mt5_bridge.py:173–197`, `:288–300`).
- Retcodes 10004–10045 have readable messages, including 10027 with the Algo Trading hint.

ISSUES

F4 (disconnect mid-order, retcode 10012), F7, F8, F16, F24, F30. The MetaTrader5 package offers no per-call timeout: a hung `order_send` holds both locks, so everything else, including closes, waits. The UI must show "sending" and must not offer a resend.

6.3 Time handling

VERIFIED

- Charts, tables and VWAP use server-time epochs consistently (`lib/format.ts:20–22`, `lib/indicators.ts:118–137`).
- News events are entered in local time and stored as UTC (`components/SettingsPage.tsx:244–255`).
- The default reset mode follows the broker's own wall clock, so broker DST shifts are handled.

ISSUES

F1 (re-anchoring, including the IANA fall-back hour), F3 (late anchor), F14 (weekends, quiet periods, silent failures), F31 (clock drift).

6.4 Architecture

The layering is clean: a pure engine, a thin service and a single MT5 module. The frontend uses one pattern for shared state (external stores read with `useSyncExternalStore`) and polls REST for history and the journal. Issues: duplicated maths (F21, F22) and large files (F27). **Cost of extending today:** a new indicator touches 2 files; a risk rule 3, inside a 140-line `evaluate`; a drawing tool about 7, including backend validation; a new order type about 10, because the pending-type map is copied in four places.

6.5 Performance

SQLite writes stay off the tick path (journal per action, risk state only when it changes, drawings debounced by 600 ms). Candle payloads are bounded (at most 5,000 bars) and the stream sends only changed keys per client. Issue: F29.

6.6 Security

CONTROL	WHERE	HOW VERIFIED	RESULT
Bind to 127.0.0.1 only	`config.py:37` `__main__.py:16–22`	Read; Settings probe (S1)	Default is loopback, but any address is accepted (F12)
Host allow-list (DNS rebinding)	`api.py:121`	Probe S1; test_api.py:65-66	Works for HTTP and WebSocket
Browser origin (CORS)	`api.py:115–120` `config.py:87–92`	Preflight probe (S1)	Foreign origins refused; dev origins :5173 allowed in production (F19)
WebSocket origin	`api.py:349–356`	test_stream.py:180-183; code	Foreign origin closed (1008); no Origin header allowed (non-browser clients)
CSRF header on state changes	`api.py:124–128`	Probe S1; tests	POST/PUT/PATCH/DELETE without `X-Requested-With: trading-ui` → 403
Credentials never logged or sent	`config.py:29–31` `mt5_bridge.py:239–249`	test_mt5_bridge.py:110-117; every response model read	SecretStr; absent from all responses; not logged

- **Input validation:** Pydantic validates every body (symbol 1–32 characters, volume above 0, prices above 0, deviation up to 1000, comment up to 24, journal note up to 2000, up to 500 drawings with text up to 500). Infinite floats are accepted but harmless: the broker rejects them and the engine checks volume and entry.
- **SSH tunnel:** works as documented. Add `ExitOnForwardFailure=yes` and `ServerAliveInterval`. Don't run the Mac mock on port 8000 at the same time: the tunnel would silently fail to bind and the browser would keep talking to MOCK, with only the badge to show it. A different local port must be added to `CORS_ORIGINS` on Windows, or the WebSocket is refused. On a VPS, use key-only SSH.

6.7 Testing

Strong unit coverage of the risk rules and the API against the mock; gaps where it matters most (F24, F25). The coverage map and the list of mock-only passes are in Appendix A; mock-versus-MT5 differences in Appendix B.

6.8 Maintainability and developer experience

Python type hints are good; the TypeScript compiler runs in strict mode with `noUncheckedIndexedAccess`. `api/types.ts` mirrors `schemas.py` by hand: every model was compared and **no drift was found**, but nothing enforces it, and `StreamRisk.risk` is an untyped `dict`. One `as unknown as` cast in `components/SettingsPage.tsx:73`. Observability and Windows readiness: F23, F26 and the checklist.

7 Findings

Ranked by severity, then by impact. Effort: S small, M several modules or a decision, L multi-step. Probe references (P1–P6, B1–B4, S1) point to section 8.

ID	AREA	ISSUE	EVIDENCE (FILE:LINES)	IMPACT	SEVERITY	EFFORT	RECOMMENDED FIX

F1	Risk · time	**Day anchor can reset mid-day.** Any change of day key re-anchors the day at the current balance and equity. Three triggers: editing the reset time or timezone in Settings, a server-time tick with `time = 0`, and the repeated hour when an IANA reset timezone falls back (the key goes back, then forward again).	`risk.py:201–225`
			`mt5_bridge.py:367–369`
			`SettingsPage.tsx:32–33`
				Today's losses are forgotten. Probe P1: down 4% at 13:00 server time, reset changed to 14:00 → daily floor drops from $95,000 to $91,200, and it re-anchors again at 14:00.	● **Critical**	S	Only let the day key move forward; reject tick times ≤ 0 or far from the expected server clock; apply reset-schedule edits from the next reset; flag them in Settings.

F2	Risk · account	**Risk state not tied to the account.** Saved risk state (day anchor, high-water mark, observed initial balance) has no account identity, and the bridge never checks which account the terminal is logged into.	`risk.py:84–93`
			`risk_service.py:38–40`
			`mt5_bridge.py:288–300`
			`AccountBar.tsx:59–73`
				Probe P3: same-day switch from a $10k to a $100k account → daily floor $9,500, drawdown floor $9,000. Hits demo → FundingPips and Phase 1 → Phase 2; the app also trades whatever account the terminal shows. The badge can't tell MetaQuotes-Demo from a demo-type prop account.	● **Critical**	M	Store `login@server` in the state; on a change, re-anchor, reset the high-water mark and observed balance, and block new orders until the initial balance is confirmed; the health check verifies login and server and fails closed; show server and masked login in the header.

F3	Risk · time	**Late day anchor uses current values.** When the anchor is recorded late (app not running at the reset), it takes the current balance and equity, not the values at the reset. With a VM that is off overnight this is the normal daily path.	`risk.py:213–227`
			`risk.py:42`
			`AccountBar.tsx:136`
				Probe P6: app started at 13:00 after a 3% loss taken in the terminal → daily floor $92,150 instead of FundingPips' $95,000. Overnight floating profit and weekend gaps also drop out of the base.	● **Critical**	M	Rebuild the day-start balance from deals since the reset (`history_deals_get`); use the higher of that and the observed balance and equity; allow a manual day-start entry that can only tighten; keep the late flag.

F4	Order path	**Unknown outcomes shown as failures.** Outcomes that are actually unknown are reported as plain failures and the dialog stays open with the same button: `order_send` returning None or an IPC error (503), retcode 10012 timeout (409), a database error after a fill (500), a browser network error.	`mt5_bridge.py:311–324`
			`mt5_bridge.py:483–493`
			`trading.py:60–74`
			`Confirm.tsx:96–107`
			`api/client.ts:64–66`
				Probes B3 and B4: the position was open while the API returned 500 (no journal row) or 503 (journal says error). Clicking the still-open button opens a second position.	● **Critical**	M	Add an unknown-outcome path: journal status `unknown`, error `kind: "unknown"`, dialog closes with "check Positions before retrying"; tag each order's comment `pd-<journal id>` and reconcile after a reconnect; journaling after a send can never fail the response.

F5	Order path · UI	**One-click double submit.** One-click mode runs the order immediately with no in-flight guard, so a double-click or key repeat sends two orders. The server has no duplicate protection.	`Confirm.tsx:58–60`
			`OrderPanel.tsx:232–281`
			`OrderPanel.tsx:388–400`
				An unintended duplicate position. Risk limits still apply, but see F8.	● **Critical**	S	In-flight guard and a disabled button while sending; optional `client_order_id` that the server de-duplicates.

ID	AREA	ISSUE	EVIDENCE (FILE:LINES)	IMPACT	SEVERITY	EFFORT	RECOMMENDED FIX

F6	Never-block rules	**Closing depends on the full context.** Close, cancel, flatten and stop-tightening all build the full risk context first (account, positions, orders, server-time tick) before the engine's never-block short-circuit runs.	`risk_service.py:94–100`
			`risk_service.py:46–64`
			`risk.py:342–343`
			`trading.py:152–169`
				Probe B2: with `SERVER_TIME_SYMBOL` missing on the broker, closing a position returns 404 and Flatten all closes nothing.	● **High**	S	Short-circuit close and cancel before building the context; tightening needs only the position and `calc_profit`; add a test for each failing dependency.

F7	MT5 compatibility	**Empty results may come back as None.** Some MetaTrader5 builds return None with `last_error` (1, 'Success') for empty positions, orders or deals (the official docs' example shows this). The bridge treats that as an error, and the stream snapshot is one try-block.	`mt5_bridge.py:311–324`
			`mt5_bridge.py:411–470`
			`stream.py:124–152`
			`mt5_mock.py:594–621`
				Probe B1: `GET /positions` and every order return 502; the stream sends only account and status (no ticks, candles or positions). Fails closed, but the app is unusable. Plausible: verify on Windows.	● **High**	S	Treat None + code 1 as empty only after `positions_total()` / `orders_total()` confirm zero (fail closed otherwise); give each snapshot section its own try-block.

F8	Lock · timing	**Just-sent orders can be invisible.** Real MT5 updates positions and orders asynchronously after `order_send` returns, so the next locked check can miss the order just sent. The concurrency test passes because the mock updates instantly.	`trading.py:51–76`
			`risk_service.py:46–55`
			`test_trading_api.py:174–180`
				Max positions, max lots and the worst case can be exceeded by orders placed in quick succession (see F5). Plausible.	● **High**	S	Keep a short-lived list of in-flight exposures (request + `result.order`) until they appear or about 10 s pass; include them in the context.

F9	Risk rule	**SL override bypasses the per-trade cap.** `allow_no_sl` is sent by the client (and honoured in one-click mode). It skips the 1% per-trade cap, and the worst case counts the new position as $0. No-SL positions opened elsewhere count only their floating P/L.	`risk.py:397–402`
			`risk.py:423–451`
			`OrderPanel.tsx:358–363`
			`schemas.py:259`
				Probe P4: 5 lots with no stop loss → allowed; trade risk not computed; worst-case equity equals the balance.	● **High**	S	Your decision (tightens): honour the override only when a server-side setting enables it, never in one-click mode; or price the no-SL leg at the day's remaining room.

F10	Risk rule	**Initial balance defaults to first seen.** If Initial balance is blank, the drawdown base is the first balance the app ever observed, stored permanently.	`risk.py:217–218`
			`risk.py:231–236`
			`AccountBar.tsx:141–146`
				First start at $95k on a $100k account → static drawdown floor $85,500 instead of $90,000; trading is allowed with only an asterisk.	● **High**	S	In live mode, block new orders until Initial balance is set, or derive it from the account's first deposit deal.

F11	Order path	**No price or volume normalisation.** Prices and volumes are never normalised on the server (tick size, volume step, min/max). The SL/TP editor and the lots field send raw input; prices are rounded to digits, not tick size. The mock silently rounds pending prices and ignores tick size.	`mt5_bridge.py:520–576`
			`BottomPanel.tsx:124–139`
			`OrderPanel.tsx:135–137`
			`lib/trading.ts:23–25`
			`mt5_mock.py:842`
				Live-only rejections (10014, 10015, 10016), possibly while moving a protective stop at a critical moment.	● **High**	S–M	Normalise in `trading.py` before the risk check, so the checked values are the sent values; floor volumes to the step; make the mock reject unnormalised values.

F12	Security	**Bind address not restricted.** `APP_HOST` accepts any address. Beyond the Host, Origin and header checks (which a non-browser client passes trivially) there is no authentication.	`config.py:37`
			`__main__.py:16–22`
			`api.py:121–128`
				`APP_HOST=0.0.0.0` on a VPS would expose an order-placing API to the network. Probe S1: the setting is accepted.	● **High**	S	Refuse to start unless the bind address is loopback.

ID	AREA	ISSUE	EVIDENCE (FILE:LINES)	IMPACT	SEVERITY	EFFORT	RECOMMENDED FIX

F13	Risk maths	**NaN or zero P/L passes the checks.** A NaN or zero result from `order_calc_profit` passes every check: `max(0.0, nan)` is 0, and a NaN worst case never compares below a floor.	`risk.py:322–329`
			`risk.py:424–458`
			`mt5_bridge.py:472–475`
				Fails open if the terminal can't price a symbol (for example a missing conversion rate). Not observed; cheap to close.	● **Medium**	S	Treat a non-finite result, or zero for a non-zero price distance, as a violation.

F14	Time	**Server time from one symbol's ticks.** Server time is the last tick of one symbol, so it stops at weekends and lags in quiet periods. Supervisor errors (for example a wrong `SERVER_TIME_SYMBOL`) are logged at DEBUG only.	`mt5_bridge.py:367–369`
			`api.py:72–78`
			`mt5_mock.py:322–325`
				Weekend crypto trades count against Friday's day; Monday's anchor is taken after any gap; anchor failures are silent.	● **Medium**	M	Learn the server's UTC offset from fresh ticks and save it; server time = UTC + offset; log failures at WARNING and show them in `/health`.

F15	Execution	**Slippage unchecked under Market execution.** Under Market execution the request price and deviation are ignored; there is no stale-tick guard and no risk check after the fill. The mock simulates Instant-execution requotes instead.	`trading.py:102–108`
			`mt5_bridge.py:513–532`
			`mt5_mock.py:807–810`
				Slippage can push the actual risk past 1% without any warning.	● **Medium**	M	Read `trade_exemode`; block new orders on ticks older than a few seconds; after a fill, recompute risk at the fill price and warn and journal if it exceeds the cap.

F16	Execution	**Pending orders hard-code RETURN filling.** Pending orders always use `ORDER_FILLING_RETURN` (market orders and closes use the symbol's flags). The mock never validates the filling mode.	`mt5_bridge.py:495–500`
			`mt5_bridge.py:558`
			`mt5_mock.py:445`
				Possible 10030 rejection of every pending order. Plausible: checklist step C2 settles it.	● **Medium**	S	Verify on the demo; if rejected, choose the pending filling mode from the symbol's flags.

F17	Exposure	**Some order types ignored in exposure.** Orders other than the four basic pending types (for example stop-limit orders placed from the terminal or the phone) are dropped from exposure.	`mt5_bridge.py:431–448`	Positions, lots and the worst case are undercounted.	● **Medium**	S	Count every order type in the risk context (no API change needed).

F18	Worst case	**Worst case ignores swap and gaps.** The worst case ignores accumulated swap and has no allowance for gaps or slippage through a stop.	`risk.py:322–326`
			`risk.py:439–449`
			`risk_service.py:50`
				Probe P5: −$400 of swap not counted. Several nights of gold swap is material against a 5% daily limit.	● **Medium**	S	Add `min(0, swap)` per position; optional gap buffer setting (tightening).

F19	Security	**Dev origins trusted in production.** The Vite dev origins (`localhost:5173`, `127.0.0.1:5173`) are trusted for CORS and WebSockets in production by default.	`config.py:40`
			`config.py:87–92`
			`api.py:347–355`
				Any page served on port 5173 (another local Vite project) can call the trading API through the tunnel. Probe S1: the preflight from :5173 is allowed.	● **Medium**	S	Make the dev origins opt-in; default to the app's own origin only.

F20	Settings	**Loosening not fully flagged.** Loosening isn't flagged for a lower initial balance, a change of daily-loss basis, or a reset time/zone change; nothing prevents loosening a limit in the middle of the day.	`SettingsPage.tsx:23–58`
			`SettingsPage.tsx:83–87`
			`risk_service.py:130–145`
				A limit can be loosened by accident (or on tilt) with no warning.	● **Medium**	S	Flag these changes; optionally apply loosening only from the next reset (your decision).

ID	AREA	ISSUE	EVIDENCE (FILE:LINES)	IMPACT	SEVERITY	EFFORT	RECOMMENDED FIX

F21	Duplicated maths	**Two pip-size rules.** Pip size differs between the order panel (gold 0.1; indices and crypto 1) and the chart's measuring tools (point × 10 or point).	`lib/trading.ts:16–21`
			`Chart.tsx:231–232`
			`DrawingLayer.ts:534–539`
				The long/short and price-range tools show 10–100× the pips the order panel uses for metals, indices and crypto.	● **Medium**	S	Use `pipSize(info)` everywhere.

F22	Duplicated maths	**Dialog shows estimated, not enforced, risk.** The confirmation dialog shows the UI's estimate (`tick_value` from a once-per-session symbol snapshot), not the risk the backend enforces (`order_calc_profit`).	`OrderPanel.tsx:143–146`
			`OrderPanel.tsx:255`
			`App.tsx:89–98`
				The risk shown can differ from the enforced figure on JPY, cross and EUR-quoted symbols.	● **Medium**	S	Show the backend's `trade_risk` in the dialog; refresh symbol specs periodically.

F23	Observability	**Journal and logs incomplete.** The journal stores the requested volume, not the filled one, and doesn't record risk-check errors. Logs go to stderr only.	`trading.py:53–74`
			`__main__.py:12–15`
			`mt5_bridge.py:485`
				After a restart, an incident is hard to reconstruct.	● **Medium**	S	Journal filled volume, price and deal plus the `pd-<id>` tag; add a rotating log file (standard library).

F24	Testing	**Mock hides live behaviour.** Mock behaviour hides live bugs: empty tuple for no results, DONE instead of PLACED for pending orders, silent price rounding, no filling or normalisation checks, always-fresh ticks (no weekends), terminal always connected, USD/JPY conversion only.	`mt5_mock.py:594–621`
			`mt5_mock.py:842–850`
			`mt5_mock.py:322–325`
			`mt5_mock.py:416–433`
				Tests pass in mock mode while the same paths fail live (Appendix B).	● **Medium**	M	Add a real-MT5-quirks switch to the mock and run the API suite in both modes.

F25	Testing	**Missing tests for risky paths.** No tests for never-block rules under dependency failures, account switching, day-key monotonicity and DST, late-anchor values, unknown outcomes or the rate limiter under concurrency; no component tests (OrderPanel, Confirm, one-click).	`test_risk.py:221–251` frontend tests cover lib/ only	Regressions in the riskiest code would go unnoticed (Appendix A).	● **Medium**	M	Write each fix's test first; add component tests once a test library is approved.
F26	Setup · Windows	**No git, checks or pinned dependencies.** Not a git repository; no lint or type checks; dependencies are unpinned; the Mac venv runs Python 3.14.6 while the README specifies 3.12 for Windows.	`requirements.txt:1–7` `README.md:82`	Untested dependency versions on the live machine and no way to roll back a change.	● **Medium**	S	Step 0 (git); build a lock file on Windows with Python 3.12; add checks later.

F27	Maintainability	**Large mixed-purpose files.** BottomPanel defines modal components inside hooks and shares state through a mutable object; DrawingLayer mixes painting and pointer handling; `evaluate()` is about 140 lines; the pending-type map exists in four places.	`BottomPanel.tsx:59–192`
			`DrawingLayer.ts:375–547`
			`risk.py:333–475`
			`trading.py:26–27`
				Every change carries regression risk; a new order type touches about ten files.	● **Medium**	L	Structural plan S1–S6 (section 10.5).

F28	Concurrency	**Rate-limit counter race.** The stream thread trims the rate-limit queue without holding the risk lock; the window also resets on restart.	`risk.py:295`
			`risk.py:303–311`
			`stream.py:127`
				Rarely, one extra action gets through.	● **Low**	S	Count without mutating, or take the lock.

ID	AREA	ISSUE	EVIDENCE (FILE:LINES)	IMPACT	SEVERITY	EFFORT	RECOMMENDED FIX

F29	Performance	**Poller makes redundant MT5 calls.** Two MT5 calls per symbol per poll (`symbol_info` + tick or rates): about 200 calls/s with 20 symbols and 8 charts. `getComputedStyle` is called for every volume bar.	`mt5_bridge.py:328–379`
			`stream.py:134–148`
			`Chart.tsx:66–69`
				CPU load on an emulated Windows-on-ARM VM; jank when history loads.	● **Low**	S	Cache symbols already selected in Market Watch; compute colours once.

F30	Execution	**Close deviation and retcode gaps.** Closes use a fixed 20-point deviation; retcode 10046 is unmapped; 10031 (no connection) doesn't mark the bridge disconnected.	`mt5_bridge.py:578–597` `mt5_bridge.py:58–99`	Under Instant execution, a fast market could requote a close repeatedly.	● **Low**	S	Wider deviation for closes; map the missing codes.
F31	News blackout	**Blackout clock and scope.** The blackout uses the Windows clock and doesn't apply when a pending order's price is moved.	`risk_service.py:56` `risk.py:390–394`	VM clock drift after the Mac sleeps can exceed the ±2-minute window.	● **Low**	S	Warn on clock skew against the server offset; apply the blackout to price changes (tightening).

F32	Polish	**Dead code and loose types.** Dead code (`calc_margin`, `MT5Bridge.connected`, the legacy `chart` message); `StreamRisk.risk` typed as `dict`; SL/TP-hit toasts guessed from the last price; stale stream cache shown while offline.	`mt5_bridge.py:225–227`
			`mt5_bridge.py:477–479`
			`schemas.py:240–242`
			`useTradeEvents.ts:38–42`
				Confusion and drift risk.	● **Low**	S	Remove or type them; add a schema-drift test.

8 Probe evidence

Each probe imports the real backend modules. "Engine only" drives `RiskEngine` directly; "API + mock" runs the FastAPI app in-process against `mt5_mock.py` (or a thin wrapper that imitates a real-MT5 quirk) with an in-memory database. Nothing touched the project's files or any MT5 account.

PROBE	SCENARIO	HARNESS	OBSERVED	FINDING
P1	Down 4% at 13:00 server time; reset time changed from 00:00 to 14:00 (also tried America/New_York 17:00)	Engine only	Day key 2026-10-12 → 2026-10-11; daily floor $95,000 → $91,200 (both variants)	F1
P2	A server-time tick with `time = 0`, then a normal tick	Engine only	Day key → 1970-01-01 → 2026-10-12; floor $95,000 → $91,200; flagged late	F1
P3	Same-day switch from a $10,000 to a $100,000 account	Engine only	Daily floor $9,500; drawdown floor $9,000 on the $100k account	F2
P4	Buy 5 lots with no stop loss, override set	Engine only	Allowed; trade risk not computed; worst-case equity $100,000 (= balance)	F9
P5	1-lot position carrying −$400 swap, plus a new 0.1-lot order	Engine only	Worst-case equity $99,450: stop losses counted, swap ignored	F18
P6	App first started at 13:00 after a 3% loss earlier the same day	Engine only	Daily floor $92,150 (true floor $95,000); flagged late	F3
B1	`positions_get` / `orders_get` return None + (1, 'Success') when empty	API + mock wrapper	`GET /positions` 502; `POST /orders` 502; stream snapshot contains only account and status	F7

PROBE	SCENARIO	HARNESS	OBSERVED	FINDING
B2	`SERVER_TIME_SYMBOL=EURUSD.raw` (absent); position opened outside the app	API + mock	`DELETE /positions/{t}` 404; flatten closed 0; position still open	F6
B3	Journal write fails after a successful fill	API + mock	`POST /orders` 500; 1 position open; 0 journal rows	F4
B4	`order_send` executes, then the reply is lost (IPC –10002)	API + mock	`POST /orders` 503 "Lost connection"; 1 position open; journal says error	F4
S1	Security controls with production-like defaults	API + mock	Foreign Host rejected (HTTP 400, WebSocket denied); evil.example preflight 400; :5173 preflight allowed; POST/PUT without the header 403; `APP_HOST=0.0.0.0` accepted	F12, F19

9 Top 5 changes by impact for the effort

1. **Fix the day anchor (F1 + F3).** The 5% daily loss is the easiest FundingPips rule to breach. Today the engine's floor can quietly drop below FundingPips' real floor: after one Settings edit, after one bad tick, or every morning if the VM is off overnight. F1 is small; F3 is medium.
2. **Tie risk state to the account (F2).** You will switch accounts at least twice: demo to FundingPips, then Phase 1 to Phase 2. Each switch currently carries the old account's limits over, and the app trades whatever account the terminal happens to be logged into.
3. **Make closing independent of everything else (F6).** The never-block rules exist so you can always get flat when things go wrong. Right now that depends on a EURUSD tick being available. Small change.
4. **Protect against duplicate orders (F5 + F4).** These are the two likeliest ways to open a position you didn't intend: a double-click in one-click mode, and retrying after an "error" that was actually a fill.
5. **Add real-MT5 compatibility fixes (F7 + F8).** Cheap insurance against the two behaviours most likely to differ from the mock on day one: empty results coming back as None, and positions appearing late after order_send.

10 Improvement plan

10.1 Step 0 (needs approval)

`git init`, then check `.gitignore`: it already excludes `backend/.env`, `backend/data/`, `.venv/`, `node_modules/`, `frontend/dist/`, `__pycache__/` and `*.log`. Then one baseline commit, so every later change is one reviewable commit that can be reverted.

10.2 Quick wins

Small, tighten-only, one commit each, with the test written first. After every step: `python -m pytest`, `npm test`, `npm run typecheck` and `npm run build`, results reported.

ID	CHANGE	FIXES	FILES	VERIFICATION
QW1	Day key only moves forward; reject invalid server time; reset-schedule edits apply from the next reset	F1	risk.py, risk_service.py	Tests: settings change, zero tick, DST fall-back hour
QW2	Close, cancel and stop-tightening without the full context	F6	risk_service.py, trading.py	Tests with each dependency failing (server time, positions, orders, account)
QW3	One-click in-flight guard; button disabled while sending	F5	OrderPanel.tsx	Unit test of the guard; double-click in mock mode places one order
QW4	None + code 1 → confirm with `*_total()` → empty list; isolate stream sections	F7	mt5_bridge.py, stream.py, mt5_mock.py	Tests against a None-for-empty mock
QW5	Track in-flight exposures until they appear	F8	risk_service.py, trading.py	Test with delayed position visibility in the mock
QW6	Non-finite or zero P/L fails closed	F13	risk.py	Engine tests with NaN and 0
QW7	Loopback-only bind; dev origins opt-in	F12, F19	config.py	Config tests; security probe
QW8	Count every order type in the risk context	F17	mt5_bridge.py, risk_service.py	Test with a stop-limit order
QW9	Include swap in the worst case	F18	risk.py, risk_service.py	Engine test (probe P5)
QW10	One pip-size rule everywhere	F21	Chart.tsx	Unit test; visual check in mock

ID	CHANGE	FIXES	FILES	VERIFICATION
QW11	Journal fill details and the `pd-<id>` tag; rotating log file	F23	trading.py, __main__.py	API tests
QW12	Rate-limit count without mutation	F28	risk.py	Concurrency test
QW13	Read-only MT5 diagnostics script for Windows (never calls `order_send`)	Checklist B2, B5	new script: tools/mt5_probe.py	You run it on Windows; it prints specs and behaviours

10.3 Decisions needed

FINDING	QUESTION	OPTIONS	RECOMMENDED
F2	Show the account in the header?	a) server only · b) server + last 4 digits of the login · c) keep hidden (current design, test_api.py:25)	b
F3	How to set the day start when the app starts late?	a) rebuild from deals · b) manual entry (tighten-only) · c) block new orders until confirmed	a + b
F4	Add `kind: "unknown"` to error responses and an `unknown` journal status?	Additive contract change	Yes
F5	Server-side duplicate protection with an optional `client_order_id`?	Additive contract change	Yes
F9	Keep the per-order no-SL override?	a) only if a setting allows it, never in one-click · b) remove it · c) keep as is	a
F10	Block live orders until Initial balance is set?	a) block · b) derive from the first deposit deal · c) keep as is	a or b
F20	When should a loosened limit take effect?	a) from the next reset · b) immediately, with a stronger warning	a

10.4 Contract changes to approve

The REST paths, status codes, error shapes, WebSocket messages, SQLite schema and risk defaults are treated as a contract. None of these loosens a default or a check.

CHANGE	SURFACE	TYPE
Error `kind` gains `"unknown"`	REST error shape (409/422/5xx bodies)	Additive
Journal status `unknown`	`JournalOut.status`, `types.ts`	Additive
Optional `client_order_id`	`POST /orders` body	Additive
Server name and masked login	`GET /account`, WS `account`	Additive; reverses a deliberate test
Risk-observer error, account mismatch	`/health`, WS `status`	Additive
`account_key`, `server_utc_offset`, in-flight list	`risk_state` JSON in the keyvalue table	No SQL change; old rows load with defaults
Narrower default CORS origins	config default	Behaviour change (dev origins opt-in)
F3, F9, F10, F20, F31 tightenings	Risk behaviour	Tighten only; need your OK

10.5 Structural improvements

Each one starts with tests that pin current behaviour.

S1 · Split mt5_bridge.py (616 lines)

CURRENT

One file holds errors and retcode text, mappings, connection and backoff, market data, account reads and order requests.

PROPOSED

```
backend/mt5/
  errors.py       MT5Error family, retcode messages
  connection.py   backoff, connect, health check, _call
                  (None and empty-result handling)
  market_data.py  symbols, ticks, candles, server time
  requests.py     request builders, rounding, filling mode
  bridge.py       MT5Bridge facade, same public methods
backend/mt5_bridge.py   re-exports: no import changes
```

Migration and verification: Snapshot tests of every request dict first; move one module per commit; keep the facade; run all suites after each step.

Easier afterwards: Request building can be unit-tested without a terminal; stop-limit orders live in one place; the mock's quirk mode targets connection.py.

S2 · Restructure risk.py (475 lines)

CURRENT

Settings and state models, status and every rule share one module; evaluate() is about 140 lines.

PROPOSED

```
backend/risk_models.py  RiskSettings, RiskState,
                        RiskDecision, RiskStatus
backend/risk.py         engine: never-block short-circuits
                        first, then an ordered list of rules
                        (intent, ctx, candidate) -> violations
```

Migration and verification: Keep all ~60 risk tests unchanged; add a table-driven test of the exact violations for a matrix of intents; move rules one at a time.

Easier afterwards: A new rule is one function and one test; the never-block guarantees are explicit and easy to audit.

S3 · Split BottomPanel.tsx (584 lines)

CURRENT

Four tabs plus two hooks that define components inside themselves and share state through a mutable object.

PROPOSED

```
components/bottom/
  PositionsTab.tsx   OrdersTab.tsx
  HistoryTab.tsx     JournalTab.tsx
  StopsEditor.tsx    CloseDialog.tsx   (real components)
lib/requests.ts      pure builders for modify and close
                     requests, unit-tested
```

Migration and verification: Extract the request builders with tests first, then move each tab. Component tests need @testing-library/react and jsdom: new dev dependencies, so your OK first.

Easier afterwards: Close and modify flows become testable; fewer surprises from the shared mutable state.

S4 · Split DrawingLayer.ts (841 lines)

CURRENT

Series-primitive glue, canvas painting for 15 tools and the pointer state machine in one class.

PROPOSED

```
components/drawing/
  render.ts        one painter per drawing type
  interaction.ts   pure pointer state machine ({x, y})
components/DrawingLayer.ts   thin glue, about 250 lines
```

Migration and verification: Geometry is already tested in lib/drawings.ts; add state-machine tests with synthetic events; visual check in mock mode.

Easier afterwards: A new tool is a painter plus geometry; interaction bugs are testable without the DOM.

S5 · Share the trading maths

CURRENT

Sizing, P/L labels and pip size are computed separately in the UI and the backend.

PROPOSED

```
one pipSize() used by every component
the dialog shows the backend's trade_risk
shared JSON fixtures: spec + entry + SL -> loss,
checked by both pytest and vitest
```

Migration and verification: Add the fixtures first, then switch the dialog to trade_risk, then remove the second pip rule.

Easier afterwards: Shown numbers equal enforced numbers; drift between the two sides fails a test.

S6 · Split mt5_mock.py (898 lines) and add a quirks mode

CURRENT

Price generation, account state, trading simulation and the API surface in one module, imitating an ideal terminal.

PROPOSED

```
backend/mock/
  prices.py   state.py   api.py
QUIRKS flags:
  none_for_empty        pending_returns_placed
  reject_unnormalised   validate_filling
  delayed_visibility    stale_weekend_ticks
  ipc_drop_after_send
```

Migration and verification: Split without behaviour change first; then add each quirk with a test; run the API suite in both modes.

Easier afterwards: Live-only failures (Appendix B) show up on the Mac before they reach the account.

10.6 Longer term

ITEM	BENEFIT	TRADE-OFF
ruff, mypy and eslint (with the react-hooks plugin) in a `scripts/check.sh` pre-commit hook	Catches whole classes of bugs, for example missing effect dependencies and Optional misuse	New dev dependencies; noisy at first
A `schema_version` table with ordered migrations and an automatic copy of trading.db before each	Safe schema changes on real journal data	A little custom code instead of Alembic's features
Generated TypeScript types from the OpenAPI schema (openapi-typescript)	No drift between schemas.py and types.ts	New dev dependency; optional and nullable fields can differ. No-dependency alternative: a pytest that compares field names
A lock file built on Windows with Python 3.12	Reproducible installs on the live machine	Updates become deliberate
Auto-start as a Windows service; a VPS rather than a VM	The anchor is taken at the real reset; survives reboots	VPS cost; another machine to secure
A mock-mode browser smoke test of the order flow (Playwright)	Catches UI regressions in the order path	New dev dependency; slower checks

11 Pre-live checklist (Windows, demo account)

You run this. Do it after the Critical fixes, then again before the first FundingPips session. Use 0.01 lots. Keep the demo and FundingPips databases separate with `DB_PATH` until F2 is fixed, and don't copy the Mac `trading.db`.

A · Set up the Windows machine

#	STEP	EXPECTED RESULT	DONE
A1	Install Python 3.12 x64; run `py -3.12 -c "import sys,struct;print(sys.version, struct.calcsize('P')*8)"`	Prints 3.12.x ... 64	☐
A2	`py -3.12 -m venv .venv`, activate, `pip install -r backend\requirements.txt`; then `python -c "import MetaTrader5 as m; print(m.__version__)"`	A version number, no error	☐
A3	`w32tm /query /status` (on a VM, turn on host time sync)	Synced within the last hour; clock within 1 s of an internet time source	☐
A4	Install and log in to the MT5 terminal; Tools → Options → Expert Advisors → Allow algorithmic trading; Algo Trading toolbar button green	Terminal connected; button green	☐
A5	Edit `backend\.env`: `MT5_MOCK=0`, credentials, `MT5_PATH` = the exact terminal64.exe, `SERVER_TIME_SYMBOL` = a symbol in Market Watch, `APP_HOST=127.0.0.1`, `DB_PATH` = a new file for this account (e.g. `backend\data\trading-demo.db`)	Don't copy the Mac trading.db: its risk state comes from the mock (F2)	☐
A6	`python -m backend`	Log line "Connected to MT5 terminal"; no tracebacks	☐
A7	On the Mac, with the mock backend stopped: `ssh -N -o ExitOnForwardFailure=yes -o ServerAliveInterval=15 -L 8000:127.0.0.1:8000 <user>@<host>`; open http://127.0.0.1:8000	Header badge LIVE · DEMO (never MOCK)	☐

B · Read-only checks (no orders)

#	STEP	EXPECTED RESULT	DONE
B1	Compare the header with the terminal's Trade tab	Balance, equity and margin match exactly	☐
B2	With no positions or pending orders: open the Positions and Pending tabs; watch the watchlist and a chart	Tabs load (no 502); prices and the forming candle update (F7)	☐
B3	Settings → Initial balance = account size → Save	Drawdown meter shows no asterisk	☐
B4	Open `/api/risk/status`; compare `day_key` with the date in Market Watch	Same date; terminal time minus UTC is a whole number of hours	☐
B5	For each symbol you'll trade, record digits, point, tick size, tick value, volume min/step/max, stops level, freeze level, filling mode and execution mode (QW13 script, or the terminal's Specification window)	Tick size equals point (otherwise F11 is live); note the execution mode (F15)	☐
B6	History tab, last 30 days	Shows the deposit deal; no error	☐

C · Order flow on the demo (0.01 lots)

#	STEP	EXPECTED RESULT	DONE
C1	Market buy 0.01 EURUSD, SL 20 pips, TP 40 pips, through the dialog	Retcode 10009; same ticket, SL and TP in the terminal; journal row accepted; the app's $ risk matches the terminal's loss at the SL to within cents	☐

#	STEP	EXPECTED RESULT	DONE
C2	Buy limit 30 pips below the market, with an SL	Retcode 10008 (placed). A 10030 means F16 is real	☐
C3	Drag the position's SL closer on the chart and confirm	SL moves in the terminal; TP unchanged	☐
C4	Edit the SL so the risk exceeds 1%	422 blocked; journal row blocked; terminal unchanged	☐
C5	Open 0.02, then close 0.01 from the Positions tab	0.01 remains; History shows two out deals	☐
C6	Cancel the pending order; then Flatten all with two positions and one pending order	Everything closed or cancelled; no errors	☐
C7	Six order actions within one minute	The sixth new order is blocked by the rate limit; a close still works	☐
C8	0.01 lot each on USDJPY, XAUUSD, an index and EURGBP	The app's $ risk matches the terminal for each	☐

D · Failure drills

#	STEP	EXPECTED RESULT	DONE
D1	Turn Algo Trading off; place an order	409, retcode 10027, message says to enable Algo Trading	☐
D2	Close the terminal; place an order; reopen the terminal	"MT5 disconnected · retry Ns"; the order returns 503; reconnects by itself; positions match	☐
D3	Kill the SSH tunnel; restore it	"Backend offline", then state resyncs; no duplicate orders	☐
D4	Restart the backend during the day	Day start (meter tooltip) and journal unchanged	☐
D5	Switch the terminal to another account (or change the .env)	Today nothing stops trading (F2). After the fix: new orders blocked until confirmed	☐
D6	Trade manually in the terminal, then start the app later the same day	Compare the daily floor with the FundingPips dashboard. Today it will be too low (F3)	☐

E · FundingPips rules

#	STEP	EXPECTED RESULT	DONE
E1	From the FundingPips dashboard and terms: daily reset time and timezone, daily-loss basis, max-loss type, news rule, weekend holding and crypto rules, commission per lot, and whether trading through the MT5 API ("expert" trades) is allowed	Settings match these rules or are tighter	☐

12 What could not be determined

MT5 behaviour to verify on Windows

- Whether this MetaTrader5 build returns None or an empty tuple for empty positions, orders and deals (checklist B2, QW13).
- How quickly `positions_get` shows a position after `order_send` returns (C1; QW5 makes it moot).
- How `copy_rates_from` treats an integer timestamp: scroll back on a chart and look for gaps or overlaps at page joins.

- Whether `symbol_info_tick` returns a zero tick for a symbol just added to Market Watch (QW13).
- Execution mode, filling flags, stops and freeze levels, tick sizes and symbol suffixes on the FundingPips server (B5, A5).
- The server's timezone and DST rule (B4).
- Whether the MetaTrader5 package supports Python 3.14 (irrelevant if Windows uses 3.12).

FundingPips rules to confirm (checklist E1)

- Daily reset time and timezone (server midnight or CE(S)T) and the daily-loss base (balance, or the higher of balance and equity).
- Whether trading through the MT5 API, recorded as "expert" trades, is permitted.
- Whether challenge and funded accounts report as demo or real (this changes the header badge).
- Weekend holding, weekend crypto trading and news-trading windows for funded accounts.
- Commission per lot and swap for your account type.

Appendix A · Test coverage map

AREA	COVERED	NOT COVERED	PASSES ONLY BECAUSE OF THE MOCK
Risk engine (risk.py)	Every rule, with boundaries: commission, basis, trailing drawdown, news window, rate limit, modifications, status levels	Day key going backwards; settings-change re-anchor; DST; late-anchor values; account switch; NaN P/L	—
Order API (trading.py)	Accept, block, reject and journal; pending lifecycle; partial and full close; flatten; rate limit versus closes; concurrency	Never-block with failing dependencies; unknown outcomes; journal failure; normalisation	Concurrency (positions appear instantly); pending DONE; requote semantics
Bridge (mt5_bridge.py)	Account, symbols, candle paging, round trip, rejection text, backoff, credentials not logged	None for empty; filling choice; IPC failure during `order_send`; 10012	Empty-tuple results
Stream (stream.py)	Snapshot, diffing, subscription changes, unknown symbols, dead clients, origin check	A partial failure inside a snapshot; stale ticks	Always-fresh ticks
Security	Host header, CSRF header, CORS preflight, WebSocket origin	Non-loopback bind; dev origins in production	—
Frontend	lib/: indicators, trading maths, drawings, drawing store, trade lines, candles, formatting	Components and hooks: OrderPanel, Confirm and one-click, BottomPanel dialogs, useTradeLines drop → request body, stream reconnect	—

Appendix B · Mock versus real MT5

BEHAVIOUR	MT5_MOCK.PY	REAL METATRADER5 (EXPECTED)	WHAT IT HIDES
Empty positions, orders, deals	Empty tuple	Build-dependent; may be None + (1, 'Success')	F7
Pending order result	10009 DONE	10008 PLACED	Untested success path
Price normalisation	Pending price silently rounded; SL/TP unchecked	Unnormalised prices rejected (10015, 10016)	F11
Filling mode	Never validated; FOK and IOC flags	10030 if unsupported	F16
Execution mode	Requote when price moved more than the deviation (Instant)	FundingPips likely Market: price and deviation ignored, slippage possible	F15
Position visibility	Immediate	Updated asynchronously after `order_send`	F8
Server time and ticks	Always fresh, 24/7	Stale at weekends and in quiet periods; can be zero for a just-selected symbol	F1, F14
Terminal connection	Connected unless the test takes it down	Terminal up but cut off from the trade server	Stale-data paths
Currency conversion	USD and JPY only	Any pair, via the terminal	Cross and EUR-index risk maths
Swaps, commission, stop-out, gaps	Not simulated	Present	F18

BEHAVIOUR	MT5_MOCK.PY	REAL METATRADER5 (EXPECTED)	WHAT IT HIDES
`order_send` failure	Never None while connected	Can return None (IPC drop, invalid request) or 10012	F4

Appendix C · Large files

FILE	LINES	WHAT IS MIXED INSIDE
`backend/mt5_mock.py`	898	Price generator, account state, trading simulation, API surface
`frontend/src/components/DrawingLayer.ts`	841	Primitive glue, canvas painting for 15 tools, pointer state machine, keyboard
`backend/mt5_bridge.py`	616	Errors and retcode text, mappings, connection and backoff, market data, account, order requests
`frontend/src/components/BottomPanel.tsx`	584	Four tabs, two modal hooks with inline components, CSV export
`backend/risk.py`	475	Settings and state models, status, the rules (evaluate is about 140 lines)
`frontend/src/components/Chart.tsx`	452	Chart lifecycle, history paging, indicators, drawings wiring, legend
`frontend/src/components/ChartTools.tsx`	440	Drawing toolbar, magnet and sync toggles, indicator menu, layout definitions
`frontend/src/components/OrderPanel.tsx`	404	Form state, sizing maths, risk check, draft sync, confirmation